

Design Pattern

→ Design Patterns are the well proved solution of commonly occurring problems in software design.

→ Design Patterns represents an idea.

Categories:

① Creational Design Pattern:

Example - Factory Pattern, Builder Pattern, Singleton Pattern etc.

② Structural Design Pattern:

Example - Proxy Pattern, Adaptor Pattern

③ Behavioral Design Pattern:

Example - Observer Design Pattern, Iterator Pattern etc.

Singleton Design Pattern:

The Singleton Design Pattern ensures that a class has only one instance.

Steps:

1. Private constructor
2. Create object with help of public static method.
3. Create variable to store object that is private.

① Lazy way:

```
public class Singleton {  
    private static Singleton singleton;  
    private Singleton() {  
    }  
    public static Singleton getSingleton() {  
        if (singleton == null) {  
            singleton = new Singleton();  
        }  
        return singleton;  
    }  
}
```

```

Public class SingletonDesignPatternExample {
    Public static void main(String[] args) {
        Singleton singleton1 = Singleton.getSingleton();
        Singleton singleton2 = Singleton.getSingleton();

        System.out.println(singleton1);
        System.out.println(singleton2);
    }
}

```

} Same fully qualified Path

② Eager way:

```

Public class Singleton {
    Private static Singleton singleton = new Singleton();
    Private Singleton() {
    }
    Public static Singleton getSingleton() {
        return singleton;
    }
}

```

```

Public class SingletonDesignPatternExample {
    Public static void main(String[] args) {
        Singleton s1 = Singleton.getSingleton();
        Singleton s2 = Singleton.getSingleton();

        System.out.println(s1);
        System.out.println(s2);
    }
}

```

} Same fully qualified Path

③ Synchronized Way:

```
public class Singleton {  
    private static Singleton singleton;  
    private Singleton() {  
  
    }  
    public static Singleton getSingleton() {  
        if (singleton == null) {  
            synchronized (Singleton.class) {  
                if (singleton == null) {  
                    singleton = new Singleton();  
                }  
            }  
        }  
        return singleton;  
    }  
}
```

Q. How to break Singleton Design Pattern?

Using Reflexion API:

```
public class Singleton {  
    private static Singleton singleton;  
    private Singleton() {  
  
    }  
    public static Singleton getSingleton() {  
        if (singleton == null) {  
            synchronized (Singleton.class) {  
                if (singleton == null) {  
                    singleton = new Singleton();  
                }  
            }  
        }  
        return singleton;  
    }  
}
```

```

Public class BreakSingleton {
    Public static void main (String[] args) throws Exception {
        Singleton S1 = new Singleton().getSingleton();
        System.out.println(S1);

        Constructors < Singleton > constructors = Singleton.class.getDeclaredConstructors();
        constructors.setAccessible(true);
        Singleton S2 = constructors.newInstance();
        System.out.println(S2);
    }
}

```

Solution 1:

If object is present → throw Exception from inside constructor.

```

Public class Singleton {
    Private static Singleton singleton;

    Private Singleton() {
        if (singleton != null) {
            throw new RuntimeException("You are trying to break Singleton Pattern!");
        }
    }

    Public static Singleton getSingleton() {
        if (singleton == null) {
            synchronized (Singleton.class) {
                if (singleton == null) {
                    singleton = new Singleton();
                }
            }
        }
        return singleton;
    }
}

```


Solution 2:

```
public enum Singleton {
```

```
    INSTANCE
```

```
}
```

```
public class Solution2 {
```

```
    public static void main (String[] args) {
```

```
        Singleton s1 = Singleton.INSTANCE;
```

```
        System.out.println(s1);
```

```
        Constructor <Singleton> constructor = Singleton.class.getDeclaredConstructor();
```

```
        Set constructor.setAccessible(true);
```

```
        Singleton s2 = constructor.newInstance();
```

```
        System.out.println(s2);
```

```
    }
```

```
}
```

Exception

Factory Design Pattern:

When there is superclass and multiple subclasses and we want to get object of subclasses based on input and requirement, then we create factory class which takes the responsibility of creating object of class based on input.

Example:

```
interface Employee {  
    int Salary();  
}
```

```
Public class AndroidDeveloper extends Employee {  
    @Override  
    Public int Salary() {  
        System.out.println("Getting android developers salary");  
        return 50000;  
    }  
}
```

```
Public class WebDeveloper implements Employee {  
    @Override  
    Public int Salary() {  
        System.out.println("Getting web developers salary");  
        return 40000;  
    }  
}
```

```
Public class EmployeeFactory {  
    Public static Employee getEmployee(String empType) {  
        if (empType.trim().equalsIgnoreCase("ANDROID DEVELOPER")) {  
            return new AndroidDeveloper();  
        }  
        else if (empType.trim().equalsIgnoreCase("WEB DEVELOPER")) {  
            return new WebDeveloper();  
        }  
        else {  
            return null;  
        }  
    }  
}
```



```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Employee employee1 = EmployeeFactory.getEmployee("ANDROID DEVELOPER");
```

```
        int sal = employee1.salary();
```

```
        System.out.println(sal);
```

```
        Employee employee2 = EmployeeFactory.getEmployee("WEB DEVELOPER");
```

```
        int sal2 = employee2.salary();
```

```
        System.out.println(sal2);
```

```
    }  
}
```

Abstract Design Pattern:

```
interface Employee {
```

```
    int salary();
```

```
    String name();
```

```
}
```

```
public class AndroidDeveloper implements Employee {
```

```
    @Override  
    public int salary() {
```

```
        return 50000;
```

```
    }  
    @Override  
    public String name() {
```

```
        return "Android Developer";
```

```
    }
```

```
}
```

```
public class WebDeveloper implements Employee {
```

```
    @Override
```

```
    public int salary() {
```

```
        return 40000;
```

```
    }
```

```
    @Override
```

```
    public String name() {
```

```
        return "Web Developer";
```

```
    }
```

```
}
```

```
Public class EmployeeFactory {
```

```
    Public static Employee getEmployee (EmployeeAbstractFactory factory) {
```

```
        return factory.createEmployee();
```

```
    }
```

```
}
```

```
Public abstract class EmployeeAbstractFactory {
```

```
    Public abstract Employee createEmployee();
```

```
}
```

```
Public class AndroidDevFactory extends EmployeeAbstractFactory {
```

```
    @Override
```

```
    Public Employee createEmployee() {
```

```
        return new AndroidDeveloper();
```

```
    }
```

```
}
```

```
Public class WebDevFactory extends EmployeeAbstractFactory {
```

```
    @Override
```

```
    Public Employee createEmployee() {
```

```
        return new WebDeveloper();
```

```
    }
```

```
}
```

```
Public class Main {
```

```
    Public static void main (String[] args) {
```

```
        Employee e1 = EmployeeFactory.getEmployee (new AndroidDevFactory());
```

```
        e1.name();
```

```
        e1.Salary();
```

```
        Employee e2 = EmployeeFactory.getEmployee (new WebDevFactory());
```

```
        e2.name();
```

```
        e2.Salary();
```

```
    }
```

```
}
```


Builder Design Pattern:

While creating object when object contain many attributes there are many problem exists:

1. we have to pass many attributes to create object.
2. some parameters might be optional.
3. factory class takes all responsibilities for creating object. if the object is heavy then all complexity is the part of factory class.

So, in Builder Design Pattern we will create object step by step and finally return final object with desired values of attributes.

Example:

```
class User {  
    private String name;  
    private int age;  
    private String country;  
    private String phone;  
  
    // Private Constructors  
    private User (Builder builder) {  
        this.name = builder.name;  
        this.age = builder.age;  
        this.country = builder.country;  
        this.phone = builder.phone;  
    }  
  
    public void showUser() {  
        System.out.println("name + ", " + age + ", " + country + ", " + phone);  
    }  
  
    // Static Builder class  
    public static class Builder {  
        private String name;  
        private int age;  
        private String country;  
        private String phone;  
  
        public Builder age(int age) {  
            this.age = age;  
            return this;  
        }  
    }  
}
```

```
Public Builder (String name) {
```

```
    this.name = name;  
}
```

```
Public Builder country (String country) {
```

```
    this.country = country;
```

```
    return this;
```

```
}
```

```
Public Builder phone (String phone) {
```

```
    this.phone = phone;
```

```
    return this;
```

```
}
```

```
Public User build () {
```

```
    return new User (this);
```

```
}
```

```
}
```

```
Public class Main {
```

```
    Public static void main (String[] args) {
```

```
        User user = new User. Builder ("John")
```

```
            .age (25)
```

```
            .country ("India").
```

```
            .phone ("8101541092")
```

```
            .build ();
```

```
        User.show Users ();
```

```
    }
```


Prototype Design Pattern:

The concept is to copy an existing object rather than creating a new instance from scratch, because creating new object may be costly.

This approach saves costly resources and time, especially when object creation is a heavy process.

```
public class NetworkConnection implements Cloneable {
```

```
    private String ip;
```

```
    private String importantData;
```

```
    public String getIP() {
```

```
        return ip;
```

```
    }
```

```
    public void setIP(String ip) {
```

```
        this.ip = ip;
```

```
    }
```

```
    public String getImportantData() {
```

```
        return importantData;
```

```
    }
```

```
    public void setImportantData(String importantData) {
```

```
        this.importantData = importantData;
```

```
    }
```

```
    public void loadImportantData() {
```

```
        this.importantData = "Important Data";
```

```
    }
```

```
    @Override
```

```
    public String toString() {
```

```
        return this.ip + " : " + this.importantData;
```

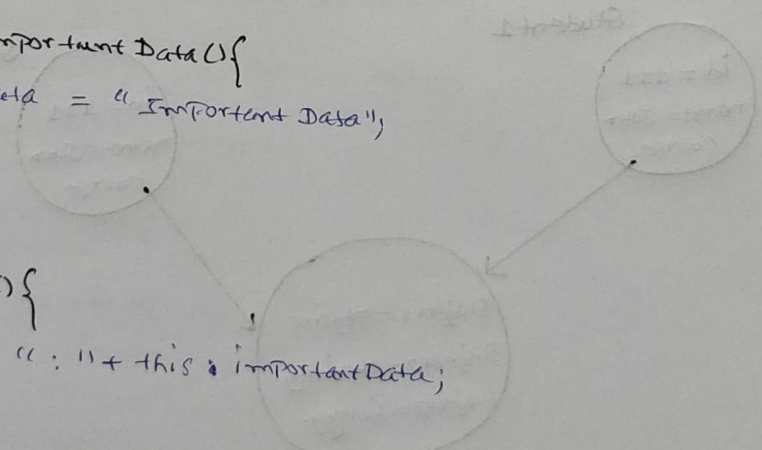
```
    }
```

```
    @Override
```

```
    protected Object clone() throws CloneNotSupportedException {
```

```
        return super.clone();
```

```
    }
```



```
Public class Main {
```

```
Public static void main (String[] args) {
```

```
System.out.println ("Creating Object using Prototype Design");
```

```
NetworkConnection networkConnection = new NetworkConnection ();
```

```
networkConnection.setIP ("192.168.10.2");
```

```
networkConnection.loadImportantData ();
```

```
System.out.println (networkConnection );
```

```
NetworkConnection networkConnection2;  
try {
```

```
networkConnection2 = (NetworkConnection) networkConnection.clone();
```

```
} System.out.println (networkConnection2);
```

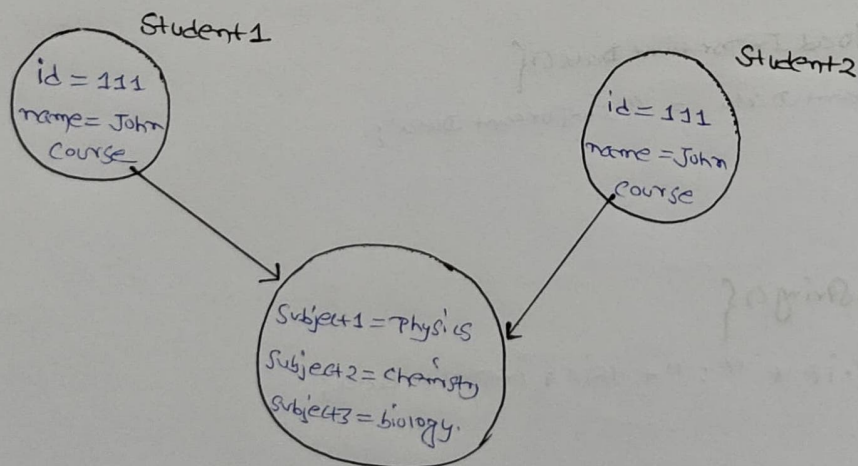
```
catch (CloneNotSupportedException e) {
```

```
e.printStackTrace();
```

```
}  
}
```

Shallow Copy:

Shallow Copy creates a new object but does not duplicate referenced objects, instead it copies their references. As a result both the original and copied objects point to the same memory location for reference fields.




```
Class Address{
```

```
String city;
```

```
Address(String city){
```

```
this.city = city;
```

```
}
```

```
}
```

```
Class Person implements Cloneable{
```

```
String name;
```

```
Address address;
```

```
Person(String name, Address address){
```

```
this.name = name;
```

```
this.address = address;
```

```
}
```

```
protected Object clone() throws CloneNotSupportedException{
```

```
return super.clone();
```

```
}
```

```
}
```

```
Public Class Test{
```

```
Public Static void main(String[] args){
```

```
Address addr = new Address("Kolkata");
```

```
Person P1 = new Person("Abhi", addr);
```

```
Person P2 = (Person) P1.clone();
```

```
P2.address.city = "Delhi";
```

```
System.out.println(P1.address.city); // Delhi
```

```
}
```

Explanation:

Both P1 and P2 point to the same address object.

Deep Copy:

A Deep Copy creates a new object along with separate copies of all referenced objects, ensuring complete independence between the original and copied object.

```
class Address {  
    String city;  
    Address (String city) {  
        this.city = city;  
    }  
}
```

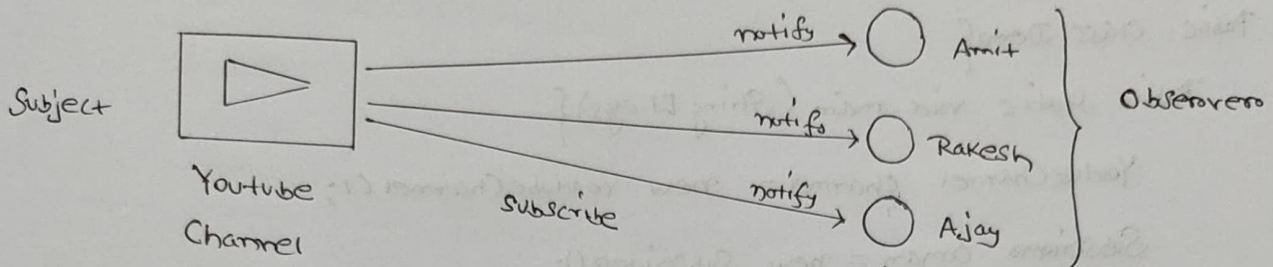
```
class Person {  
    String name;  
    Address address;  
    Person (String name, Address address) {  
        this.name = name;  
        this.address = address;  
    }  
}
```

// Deep Copy Constructor

```
Person (Person p) {  
    this.name = p.name;  
    this.address = new Address (p.address.city);  
}
```


Observer Design Pattern:

- It is a behavioral Design Pattern.
- In this when Subject changes the state all its dependent objects notified the changes.
- one to many relation.



```
interface Subject {  
    void subscribe(Observer ob);  
    void unsubscribe(Observer ob);  
    void notifyChanges()  
}
```

```
public interface Observer {  
    void notified();  
}
```

```
public class YoutubeChannel implements Subject {  
    List<Observer> subscribers = new ArrayList<>();  
    String url;  
    @Override  
    public void subscribe(Observer ob) {  
        this.subscribers.add(ob);  
    }  
    @Override  
    public void unsubscribe(Observer ob) {  
        this.subscribers.remove(ob);  
    }  
    @Override  
    public void notifyChanges() {  
        for (Observer ob : this.subscribers) {  
            ob.notified();  
        }  
    }  
}
```

Public class Subscriber implements Observer {

@Override

Public void notify() {

System.out.println("New video upload: notification");

}

}

Public class Demo {

Public static void main (String [] args) {

YoutubeChannel channel = new YoutubeChannel();

Subscriber aman = new Subscriber();

Channel.subscribe(aman);

Channel.notifyChanges();

}

}